

Derin Öğrenme Tabanlı Araç Plaka Tespit ve Okuma Sistemi

Bu rapor, Python Programlamaya Giriş dersi kapsamında verilen ödevde istenen başlıklara göre hazırlanmıştır. Çalışmada proje adı, konu, public GitHub bağlantısı, yöntem, algoritma akış şeması, uygulama tasarımı, başarı ölçütleri, teknik prototip, örnek senaryo, somut çıktılar ve APA biçiminde kaynakça yer almaktadır.

1. Proje Adı

Derin Öğrenme Tabanlı Araç Plaka Tespit ve Okuma Sistemi

2. Proje Konusu

Bu projenin amacı, bir araç fotoğrafı verildiğinde plaka bölgesini otomatik olarak tespit eden ve tespit edilen plaka üzerindeki karakterleri okuyan bir Python prototipi geliştirmektir. Sistem iki aşamalı tasarlanmıştır: ilk aşamada YOLO tabanlı nesne tespitiyle plaka bölgesi bulunur, ikinci aşamada ise kırılan plaka görüntüsü EasyOCR ile okunur.

YOLO yaklaşımı, nesne tespitini tek bir sinir ağı geçişinde gerçekleştirdiği için hızlı ve gerçek zamanlı uygulamalara uygundur (Redmon et al., 2016). Plaka tanıma alanında da YOLO tabanlı yöntemlerin farklı görüntü koşullarında etkili sonuçlar verebildiği gösterilmiştir (Laroca et al., 2021).

3. GitHub Linki

GitHub public repo linki: <https://github.com/mhmtmndrn/Derin-Ogrenme-Tabanli-Arac-Plaka-Tespit-Ve-Tanima-Sistemi>

4. Yöntem

Projede plaka tespiti için Ultralytics YOLO11n modeli kullanılmıştır. Model, Roboflow Universe üzerindeki License Plate Recognition v11 veri setiyle Kaggle Notebook ortamında eğitilmiştir. Veri seti YOLO formatında indirilmiş ve eğitim sonunda en iyi model ağırlığı `best.pt` olarak alınmıştır.

Plaka okuma aşamasında EasyOCR kullanılacaktır. Tespit edilen plaka bölgesi önce kırılır, sonra gri tonlama, kontrast artırma ve eşikleme gibi ön işleme adımlarıyla OCR için hazırlanır. EasyOCR sonucunda elde edilen metin büyük harfe çevrilir ve sadece harf/rakam karakterleri kalacak şekilde temizlenir.

Projenin çalışma adımları şu şekildedir:

1. Araç fotoğrafı programa verilir.

2. Eğitilmiş YOLO11n modeli yüklenir.
3. Görsel üzerindeki plaka bölgesi nesne tespitiyle bulunur.
4. En güvenilir plaka kutusu seçilir ve plaka alanı kırpılır.
5. Kırpılan görüntü OCR için ön işlemden geçirilir.
6. EasyOCR ile plaka karakterleri okunur.
7. Metin temizlenir, güven skorları hesaplanır ve sonuçlar görsel/CSV olarak kaydedilir.

5. Algoritma Akış Şeması

Metinsel akış şeması özeti:

Araç fotoğrafı -> Görsel kontrolü -> YOLO11n ile plaka tespiti -> Plaka kırpma -> Görüntü ön işleme -> EasyOCR ile okuma -> Metin temizleme ve güven skoru -> Görsel/CSV çıktı

flowchart TD

```

A["Girdi: Araç fotoğrafı"] --> B{"Görsel dosyası okunabiliyor mu?"}
B -- "Hayır" --> C["Hata mesajı: görsel bulunamadı veya okunamadı"]
B -- "Evet" --> D["YOLO11n plaka tespit modeli yüklenir"]
D --> E["Görselde plaka kutusu tahmin edilir"]
E --> F{"Plaka tespit edildi mi?"}
F -- "Hayır" --> G["Çıktı görseline tespit edilemedi notu yazılır"]
F -- "Evet" --> H["En yüksek güvenli plaka bölgesi kırpılır"]
H --> I["Ön işleme: gri ton, kontrast artırma, eşikleme"]
I --> J["EasyOCR ile plaka karakterleri okunur"]
J --> K["Metin temizlenir ve güven skoru hesaplanır"]
K --> L{"OCR güveni yeterli mi?"}
L -- "Hayır" --> M["Sonuç düşük güven olarak işaretlenir"]
L -- "Evet" --> N["Sonuç başarılı olarak işaretlenir"]
M --> O["İşaretlenmiş görsel, kırpım ve CSV kaydedilir"]
N --> O
G --> O

```

Algoritma açıklaması ödevde istenen dört temel parçaya göre aşağıdaki gibidir:

- Girdi: Kullanıcının verdiği araç fotoğrafı ve eğitilmiş YOLO ağırlık dosyasıdır.
- İşlem: Fotoğraf okunur, YOLO modeliyle plaka kutusu tahmin edilir ve plaka bölgesi kırpılır.
- Hesaplama: Plaka kutusu güven skoru, OCR sonucu ve OCR güven skoru hesaplanır; okunan metin harf/rakam dışı karakterlerden temizlenir.
- Çıktı: İşaretlenmiş sonuç görseli, kırpılmış plaka görüntüsü ve plaka metni/güven skorlarını içeren CSV dosyası üretilir.

6. Uygulama Tasarımı

Kullanıcı programa bir araç fotoğrafı ve eğitilmiş YOLO ağırlık dosyası verir. Program fotoğrafı okur, plaka bölgesini tespit eder, plakayı kırpar ve OCR işlemini uygular.

Örnek komut:

```
python src/main.py --image samples/ornek_arac.jpg --weights models/best.pt --output outputs/demo_result.jpg --csv outputs/results.csv
```

Kullanıcının gireceği bilgiler:

- `--image` : İşlenecek araç fotoğrafının yolu
- `--weights` : Eğitilmiş YOLO model dosyasının yolu
- `--output` : İşaretlenmiş sonuç görselinin kaydedileceği yol
- `--csv` : Sonuç tablosunun kaydedileceği yol

Programın hesapladığı değerler:

- Plaka konumunu belirleyen koordinatlar
- YOLO tespit güven skoru
- OCR ile okunan ham ve temizlenmiş plaka metni
- OCR güven skoru
- Sonuç durumu: başarılı, düşük güven veya tespit edilemedi

Programın ürettiği çıktılar:

- Plaka kutusu çizilmiş işlenmiş görsel
- Kırpılmış plaka görseli
- Okunan plaka metni
- YOLO tespit güveni ve OCR güven skorunu içeren CSV tablo

7. Başarı Ölçütleri

Projenin başarılı sayılması için aşağıdaki ölçütler kullanılacaktır:

- Program verilen görseli okuyabilmelidir.
- YOLO modeli plaka bölgesini doğru veya kabul edilebilir şekilde tespit etmelidir.
- OCR aşaması plaka karakterlerini okunabilir bir metne çevirmelidir.
- En az bir örnek senaryo baştan sona çalıştırılmalıdır.
- Sonuç görsel ve CSV tablo olarak kaydedilmelidir.
- Kaggle eğitiminde validasyon/test metrikleri rapora eklenmelidir.

8. Teknik Kısım

Programın ana akışı `src/main.py` dosyasında kurulmuştur. Ana akışta komut satırı argümanları okunur, görsel dosyası kontrol edilir, YOLO modeliyle tespit yapılır, plaka kırılır, OCR uygulanır ve sonuçlar kaydedilir. Böylece ödevde istenen en az bir işlem hattı baştan sona çalıştırılmıştır.

Fonksiyonlar modüler hale getirilmiştir:

- `src/detector.py` : YOLO modelini yükler ve plaka kutularını tespit eder.
- `src/preprocess.py` : görsel okuma, plaka kırma, OCR ön işleme ve metin temizleme işlemlerini yapar.
- `src/ocr_reader.py` : EasyOCR ile plaka metnini okur.
- `src/visualize.py` : tespit sonucunu görsel üzerine çizer.
- `src/main.py` : tüm akışı komut satırından çalıştırır.

Eğitim not defteri `notebooks/plaka_model_egitimi_kaggle.ipynb` dosyasındadır. Bu defterde veri seti indirme, YOLO eğitimi, doğrulama, test ve örnek tahmin adımları bulunur.

Grafik, tablo ve ara çıktılar da üretilmiştir. Eğitim sürecinden `results.csv` , `results.png` , `confusion_matrix.png` , `BoxPR_curve.png` ve örnek tahmin görseli alınmıştır. Prototip çalıştırıldığında ayrıca `outputs/results.csv` , `outputs/demo_result.jpg` ve kırılmış plaka görüntüsü oluşturulmuştur.

9. Örnek Senaryo

1. Kullanıcı `samples/ornek_arac.jpg` dosyasını proje klasörüne ekler.
2. Kaggle eğitiminden indirilen `best.pt` dosyasını `models/best.pt` olarak kaydeder.
3. `python src/main.py --image samples/ornek_arac.jpg --weights models/best.pt --output outputs/demo_result.jpg --csv outputs/results.csv` komutunu çalıştırır.
4. Program plaka kutusunu işaretler, plaka kırımını `outputs/crops/` klasörüne kaydeder ve sonucu `outputs/results.csv` tablosuna yazar.

10. Sonuçlar

Model eğitimi Kaggle Notebook ortamında iki adet Tesla T4 GPU ile tamamlanmıştır. Eğitimde YOLO11n modeli kullanılmış, eğitim çıktıları `docs/egitim_ciktilari/` klasörüne alınmış ve en iyi model ağırlığı `models/best.pt` olarak kaydedilmiştir.

Eğitim ve test sonuçları:

- Eğitim epoch sayısı: 30
- Eğitim süresi: yaklaşık 0.749 saat
- Validasyon precision: 0.987
- Validasyon recall: 0.941
- Validasyon mAP50: 0.9685
- Validasyon mAP50-95: 0.6872

- Test mAP50: 0.9693
- Test mAP50-95: 0.6893

Eğitim çıktıları:

- docs/egitim_ciktilari/results.csv
- docs/egitim_ciktilari/results.png
- docs/egitim_ciktilari/confusion_matrix.png
- docs/egitim_ciktilari/BoxPR_curve.png
- docs/egitim_ciktilari/val_batch0_pred.jpg

Modelin validasyon ve test metrikleri, plaka tespit aşamasının başarılı çalıştığını göstermektedir.

Gerçek araç fotoğrafı ile demo sonucu:

- Kullanılan görsel: samples/ornek_arac.jpg
- YOLO tespit güven skoru: 0.7918
- OCR ile okunan plaka: 34B5592
- OCR güven skoru: 0.8989
- Demo durumu: başarılı

Demo çıktıları:

- outputs/demo_result.jpg
- outputs/crops/ornek_arac_plate_crop.jpg
- outputs/results.csv

11. Teslim Edilmesi Gereken Somut Çıktılar

Ödev dosyasında istenen somut çıktılar bu projede aşağıdaki şekilde karşılanmıştır:

- Akış şeması: docs/akis_semasi.mmd ve rapordaki mermaid akış şeması
- Algoritma açıklaması: raporun yöntem ve algoritma akış şeması bölümleri
- Çalışan ilk prototip kod: src/main.py ve modüler src/ dosyaları
- Baştan sona denenmiş örnek senaryo: samples/ornek_arac.jpg ile yapılan demo
- Tablo/grafik/ara çıktı: outputs/results.csv , docs/egitim_ciktilari/results.png , docs/egitim_ciktilari/confusion_matrix.png
- Model çıktısı: models/best.pt
- Kaynakça ve literatür kullanımı: raporda YOLO, ALPR, YOLO11, Roboflow ve EasyOCR kaynaklarına APA biçiminde yer verilmiş; ilgili bölümlerde metin içi atıf yapılmıştır.

12. Kaynakça

JaiedAI. (2024). *EasyOCR: Ready-to-use OCR with 80+ supported languages*. GitHub.
<https://github.com/JaiedAI/EasyOCR>

Laroca, R., Zanlorensi, L. A., Gonçalves, G. R., Todt, E., Schwartz, W. R., & Menotti, D. (2021). An efficient and layout-independent automatic license plate recognition system based on the YOLO detector. *IET Intelligent Transport Systems, 15*(4), 483-503. <https://doi.org/10.1049/itr2.12030>

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 779-788. <https://doi.org/10.1109/CVPR.2016.91>

Roboflow Universe Projects. (2025). *License Plate Recognition Dataset v11*. Roboflow Universe.
<https://universe.roboflow.com/roboflow-universe-projects/license-plate-recognition-rxg4e/dataset/11>

Ultralytics. (2026). *Ultralytics YOLO documentation*. <https://docs.ultralytics.com/>